

RegretGate - ControlPlane Checker

Accenture Innovation Challenge Round 2 | Problem Track 1: ControlPlane.ai

Act with confidence. Verify only when the regret is high.

Public GitHub: <https://github.com/harshalDharpure/controlplane-regretgate>

1. What this project is about

RegretGate is an action-aware AI control plane. Enterprises use generative AI across chatbots, internal copilots, and regulated decision tools. Failures often appear only AFTER the model has already acted - wrong answers, wasted compute, or privacy/policy breaches.

RegretGate sits BEFORE commit (before reply / send / refund / approve / execute). For every pending AI action it: identifies intent, estimates Expected Regret, runs a responsibility check, routes through an intervention ladder, logs an audit receipt, and can escalate to a human.

Expected Regret = $P(\text{failure}) \times \text{Impact} \times \text{Irreversibility}$ (score 0-100)

2. Three silent failure modes

- Confidently wrong - hallucinations / poor decisions with certainty (grounding signals)
- Quietly expensive - excess tokens, tool calls, retries, agent thrash
- Subtly unsafe - bias, leakage, policy violations (PII, secrets, unsafe content)

3. How it is helpful

- Business owners: irreversible actions held for proof or human review
- Risk / privacy / compliance: hard blocks, geo policy packs, full audit trail
- AI platform teams: one gate across use cases with different latency/risk appetites
- FinOps: thrash detection stops quietly expensive agent loops
- Human reviewers: HITL only when regret is high - less alert fatigue

In short: faster where it is safe, stricter where consequences matter - shifting AI safety from post-hoc postmortems to pre-commit control.

4. Prototype surfaces

Control Plane / Paste or load pending action; score, ladder, rewrite, receipts

Scenarios /scenarios One-click enterprise demos (support, copilot, decision)

Ops /ops HITL queue, audit log, metrics, feedback, policy tuning

Tech: Next.js 16, React 19, TypeScript, Tailwind, Vitest. No API keys required.

5. Prerequisites

- Node.js 20 or newer (<https://nodejs.org>)
- npm (included with Node.js)
- Git (to clone the repository)

```
node -v
```

```
npm -v
```

6. How to run the complete application

Step 1 - Clone

```
git clone https://github.com/harshalDharpure/controlplane-regretgate.git
cd controlplane-regretgate
```

Step 2 - Install dependencies

```
npm install
```

Step 3 - Start the development server

```
npm run dev
```

Step 4 - Open in browser

- Control Plane: <http://localhost:3000>
- Scenarios: <http://localhost:3000/scenarios>
- Ops: <http://localhost:3000/ops>

Step 5 - Stop server with Ctrl+C in the terminal.

7. Optional commands

```
npm run build  
npm start  
npm test  
npm run lint
```

8. Suggested demo walkthrough (3-5 minutes)

- Control Plane -> Load Safe FAQ reply -> Evaluate -> Pass instantly
- Scenarios -> Quietly expensive tool thrash -> thrash + elevated regret
- Scenarios -> Subtly unsafe PII send -> HARD BLOCK
- Scenarios -> Compounding approve spend -> Hold -> Ops HITL resolve
- Scenarios -> Policy variant EU refund -> compare US vs EU packs

9. Intervention ladder

- Near-zero -> Pass instantly
- Low -> Attach receipt
- Medium -> Soft rewrite
- High -> Hold at gate (HITL)
- Critical -> Hard block

10. Troubleshooting

- node/npm not found: install Node 20+ and restart terminal
- Port 3000 busy: `npx next dev -p 3001`
- Old UI: hard refresh `Ctrl+Shift+R` and restart `npm run dev`
- HITL empty: run a high-regret scenario first

11. Team / license

RegretGate - Accenture Innovation Challenge Round 2 - ControlPlane.ai track

Prototype code provided for challenge evaluation.